Managing AI Session Memory with Variety Engineering: Hash Gates, Memory Temperature, and Turning Points

Palle Torsson*

April 2026

Abstract

AI coding assistants accumulate session logs that grow beyond any developer's capacity to review manually. We present Claude Context Manager, a practical memory-management architecture for AI coding sessions that uses hash-gated indexing to avoid redundant reprocessing of unchanged session files—eliminating almost all redundant re-indexing on stable session corpora. The system layers three mechanisms of decreasing maturity: (1) a three-tier hash gate (mtime, content hash, full re-index) that serves as the primary efficiency mechanism; (2) a memory temperature score that triages memories by recency, connectivity, and importance for human browsing; and (3) heuristic turning-point extraction that identifies mid-session pivots and breakthroughs during clone-to-thread operations. We interpret the design through Ashby's Law of Requisite Variety, framing content hashing as a variety attenuator at layer boundaries. Evaluated on a real-world corpus of 2,434 sessions (1.0 GB) across 13 projects, the hash gate achieves a 99.96% cache hit rate on warm paths. In an ablation simulating the case where file timestamps change but content does not (e.g., after a backup or copy), the hash tier avoids full re-indexing at 85% lower cost than falling through to a full parse.[1]

1 Introduction

AI coding assistants such as Claude Code, GitHub Copilot, and Cursor generate persistent session logs as a side effect of their operation. A developer working with Claude Code for several months may accumulate hundreds of sessions across multiple projects, each recorded as a JSONL file of 10 to 50,000+ events. These logs contain valuable information—architectural decisions, debugging breakthroughs, abandoned approaches—but their volume makes them impractical to search manually.

The problem is one of *variety management*. In cybernetic terms [Ashby, 1956], the variety of the session corpus (the number of distinguishable states across all sessions) grows with every new interaction, while the developer's capacity to regulate that variety remains bounded by attention, time, and the AI's context window. When environmental variety exceeds regulatory capacity, the system fails to maintain useful organization. We call this the **variety trap**.

This paper presents Claude Context Manager, a practical memory-management architecture for AI coding sessions, designed using cybernetic reasoning about variety regulation. The system makes three contributions of decreasing maturity:

*With assistance from Claude (Anthropic). The AI contributed to code implementation and paper drafting but does not meet authorship criteria under standard academic norms.

[1]Source code: https://github.com/palletorsson/claude-context-manager

1. **Three-tier hash-gated indexing** (primary)—a content-hashing architecture that places decision gates at each memory-layer boundary, reducing arbitrary file content to a 1-bit control signal to avoid redundant reprocessing
2. **Temperature-based memory triage** (secondary)—a multi-signal scoring system that classifies memories as hot/warm/cold/frozen for human browsing and review prioritization
3. **Turning-point extraction** (exploratory)—a heuristic method for identifying mid-session pivots and breakthroughs during clone-to-thread operations

We evaluate these mechanisms on a real-world corpus with component benchmarks and an ablation study, and report qualitative observations from three months of daily use.

2 Related Work

2.1 LLM Memory and Context Management

MemGPT [Packer et al., 2023] introduces a virtual memory hierarchy allowing LLMs to manage their own context through read/write operations; **Letta** [Letta, 2024] productionizes this as an agent framework. Our system operates *outside* the LLM's context window, providing a developer-facing browsing tool rather than an in-conversation memory pager. **Zep** [Zep AI, 2024] provides memory with automatic summarization and entity extraction; unlike Zep, our system is LLM-free, using TF-IDF and Jaccard clustering. **ChatGPT Memory** [OpenAI, 2024] stores model-selected facts; our system preserves full session records and lets the developer decide what matters. **LangChain** [Chase, 2022] and **LlamaIndex** [Liu, 2022] focus on runtime context within a single application rather than cross-session corpus management.

2.2 Retrieval and Memory Augmentation

RAG systems [Lewis et al., 2020, Gao et al., 2024] retrieve documents at inference time. Our temperature scoring can be viewed as a pre-filtering layer for future RAG: hot memories would be retrieval candidates, frozen ones excluded. **MemoryBank** [Zhong et al., 2024] uses an Ebbinghaus forgetting curve for decay; our scoring uses a multi-dimensional signal (recency, connectivity, importance) rather than a single temporal function. Wang et al. [2024] survey agent memory architectures; our system is a "meta-memory" layer managing outputs of agent sessions.

2.3 Developer Session Management

Codoban et al. [2015] found developers frequently need the "why" behind changes, not just diffs. Fritz & Murphy [2010] identified "information fragments" scattered across tools; our thread files serve this role for AI sessions. Murphy et al. [2006] documented IDE context-switching; our clone feature addresses task resumption. **Cursor** [Anysphere, 2024] and **Continue.dev** [Continue, 2024] manage in-session context but provide no cross-session indexing, classification, or clustering.

2.4 Cybernetics in Software Systems

Ashby's Law [Ashby, 1956] has been applied to software architecture [Bider & Jalali, 2016], organizational design [Beer, 1979, Espejo & Reyes, 2011], and sociotechnical systems [Boisot & McKelvey, 2011]. Shannon's channel capacity theorem [Shannon, 1948] frames the context window as a bandwidth-limited channel. Our contribution is a concrete application of these ideas to AI session memory, where the engineering outcomes can be measured.

3 System Design

3.1 Architecture Overview

Claude Context Manager is a web application with a Python/FastAPI backend and Next.js frontend. It reads Claude Code's session logs (JSONL files in `~/.claude/projects/`) in read-only mode and maintains a SQLite cache database for indexed metadata. Figure 1 shows the data flow.

```
Session JSONL files (raw data layer)
        |
        | content_hash gate (Tier 2)
        v
Session Index (cache.db) -----> Concept reference counts
        |                        (cross-session amplifier)
        | sessions_hash gate
        v
Topic Clusters (topic_cache) --> Memory Temperature
        |                        (hot/warm/cold/frozen)
        | file_hash gate
        v
Memory Files (curated layer) --> Dashboard + API
```

Figure 1: System architecture. Hash gates at each layer boundary serve as decision points; concept reference counting and topic clustering surface cross-session patterns.

3.2 Three-Tier Hash Gate

The central efficiency mechanism is a three-tier gate that prevents redundant computation when session files have not changed:

Tier 1: Mtime fast-path ($O(1)$). Compare the file's filesystem modification time against the cached value. If unchanged within 1-second tolerance, skip entirely. This handles the common case of repeated API requests against stable files.

Tier 2: Hash comparison ($O(\textbf{file_size})$). If mtime differs (e.g., the file was copied, backed up, or touched), compute a SHA-256 hash and compare against the cached hash. If content is identical, update only the cached mtime and skip re-indexing.

Tier 3: Full re-index ($O(\textbf{file_size} + \textbf{events})$). Only reached when file content has actually changed. Stream the JSONL file, extract metadata (message counts, timestamps, tools used, first/last messages), compute classification and importance score, and update the cache.

The hash comparison produces a 1-bit decision—"changed" or "not changed"—which is the minimum information needed for the control decision of whether to re-index.

3.3 Memory Temperature

Memory temperature provides a single-axis triage score for **human browsing**: when a developer opens the memory dashboard, hot items surface first, frozen items can be safely ignored. The score is computed from three signals:

$$\text{score} = \underbrace{\max(0,\ 40 - d \times 1.5)}_{\text{recency}\in[0,40]} + \underbrace{\min(30,\ r \times 3)}_{\text{connectivity}\in[0,30]} + \underbrace{\min(30,\ I \times 0.3)}_{\text{importance}\in[0,30]} \tag{1}$$

where d is days since last reference, r is the cross-session reference count, and I is raw importance score. The score (0–100) maps to four labels:

- **Hot** ($\geq$75 or $\leq$7 days): actively relevant
- **Warm** ($\geq$40 or $\leq$30 days): recently relevant
- **Cold** ($\geq$15 or $\leq$60 days): aging
- **Frozen** ($<$15): candidate for archival

The current operational purpose is browsing triage. In future work (Section 7), temperature could serve as a filter for automated context injection or lifecycle management.

3.4 Session Classification and Importance Scoring

Sessions are auto-classified into four categories: **Major** ($\geq$200 messages or $\geq$1 MB), **Standard** (typical interactive sessions), **Minor** ($\leq$2 user messages and $\leq$6 total), and **Automated** (batch jobs detected by known prompt patterns).

Importance scoring (0–100) combines five signals: message volume (0–30), user engagement (0–20), tool diversity (0–15), file operations (0–10), and file size (0–10), with bonuses for continuation sessions and penalties for automated/minor sessions.

3.5 Turning-Point Extraction

When cloning a session into a resumable thread file, the system extracts not just the first user message and last assistant summary, but also **turning points**—moments where understanding shifts mid-session. Two types are detected via keyword matching:

Pivots from user messages (e.g., "actually,", "scratch that", "different approach") capture moments where the plan changed direction. **Breakthroughs** from assistant messages (e.g., "the issue was", "root cause", "found it") capture moments of discovery.

This is an explicitly heuristic approach with known limitations (Section 6), included as an exploratory feature rather than a validated contribution.

3.6 Topic Clustering

Cross-session topic detection uses TF-IDF keyword extraction from session first messages, followed by Jaccard similarity clustering. Results are cached by a composite hash of all session metadata, so the $O(n^2)$ Jaccard computation only runs when sessions are added or modified. Appendix A traces a concrete session through all stages of the pipeline.

4 Theoretical Interpretation

We frame the system design through Ashby's Law of Requisite Variety, treating this as a *design interpretation guided by cybernetic principles* rather than a formal proof or validation of the law itself. The evaluation (Section 5) tests engineering outcomes predicted by this interpretation; it does not claim to validate Ashby's law as such.

4.1 Ashby's Law as Design Heuristic

Ashby's Law [Ashby, 1956] states that a regulator must have at least as many response states as the disturbances it faces. In our system:

- **Disturbance variety**: the number of distinguishable states across all sessions (grows unboundedly)

- **Regulatory capacity**: bounded by human attention, context window (∼200K tokens), and computation time

The three-tier hash gate acts as a **variety attenuator**: it reduces incoming variety at each layer boundary. The content hash reduces arbitrary file content to 256 bits; the comparison reduces it further to 1 bit. Memory temperature reduces multi-dimensional state to four categories.

Concept reference counting and topic clustering act as **variety amplifiers**: they surface cross-session patterns that no single session contains, increasing the developer's regulatory capacity without requiring manual scanning.

This framing guided concrete design decisions—placing hash checks at every layer boundary, choosing four temperature categories rather than continuous scores for the browsing interface—but we do not claim it is the only framework that could have produced similar design choices.

4.2 Shannon's Channel Capacity

The AI's context window can be modeled as a bandwidth-limited channel [Shannon, 1948]. Memory temperature provides a basis for relevance filtering when injecting context: hot memories maximize information density within the bandwidth constraint.

5 Evaluation

5.1 Dataset

We evaluate on a real-world corpus from three months of daily Claude Code usage (Table 1).

Table 1: Evaluation corpus.

Metric	Value
Projects	13
Session files	2,434
Total corpus size	1,015.3 MB
Smallest session	1.5 KB
Largest session	192.3 MB
Median session	39.1 KB
Memory files	14

5.2 Cache Hit Rates

On a warm cache (all sessions previously indexed), the three-tier gate was evaluated across all 2,434 sessions (Table 2).

The Tier 2 (hash) gate shows 0% hits in this run because the warm cache already has correct mtimes. The hash tier activates when files are copied, backed up, or touched without content modification—a scenario tested in the ablation (Section 5.4).

5.3 Computation Savings

Benchmarks on the 20 largest session files (879 MB total) measure cost differences between cached and uncached paths (Table 3).

Table 2: Cache hit rates on warm cache ($N = 2{,}434$ sessions).

Gate Tier	Count	Percentage
Tier 1: Mtime match ($O(1)$ skip)	2,433	99.96%
Tier 2: Hash match (content same)	0	0.00%
Tier 3: Full re-index (changed)	1	0.04%
Total computation avoided	**2,433**	**99.96%**

Table 3: Computation savings: uncached vs. cached paths.

Subsystem	Uncached	Cached	Reduction
Session indexing (20 files, 879 MB)	4,292 ms	718 ms	**83%**
Topic clustering (4 projects)	3.5 ms	2.7 ms	23%
Projects discovery	0.06 ms	0.03 ms	54%

Session indexing shows the largest absolute savings. The median per-file time drops from 101.6 ms (full parse) to 15.5 ms (hash only)—a 6.5× speedup.

5.4 Ablation Study

To separate the contributions of each mechanism, we run two ablations.

5.4.1 Gate tier ablation.

We measure the cost of processing the 20 largest files (880 MB) under two strategies when mtime differs: (a) skip directly to full re-index (no hash tier), and (b) hash-check first, re-index only if content changed (Table 4).

Table 4: Gate tier ablation ($N = 20$ files, 880 MB). Cost when mtime has changed.

Strategy	Total (ms)	Median/file (ms)
(a) Mtime-only gate → full re-index	4,753	114.0
(b) Mtime + hash gate → skip if unchanged	713	16.7
Hash tier savings		**85%**

When mtime differs but content is unchanged, the hash tier avoids full re-indexing at 15% of the cost. This directly validates the Tier 2 mechanism that shows 0% hits in the warm-cache benchmark: when triggered (backup, copy, filesystem touch), it provides an 6.7× speedup over falling through to full re-index.

5.4.2 Clone extraction ablation.

We measure extraction yield across 10 large sessions, comparing three levels: (a) baseline (first message + last summary), (b) adding decisions and questions, (c) adding turning points (Table 5).

Turning-point extraction adds 182 items (+26% over the decisions+questions level). The yield is modest per session but targets a specific type of mid-session information—plan changes and root-

Table 5: Clone extraction counts by feature level ($N = 10$ sessions, median 3,071 messages each).

Extraction level	Total items	Per session
(a) First msg + last summary (baseline)	20	2.0
(b) + decisions + questions	704	70.4
(c) + turning points (full system)	886	88.6
Turning points breakdown:		
Pivots (user redirections)	32	3.2
Breakthroughs (discoveries)	150	15.0

cause discoveries—that is disproportionately valuable for task resumption. We note that this is an extraction *count*, not a precision measure; false-positive rates are not evaluated here (see Section 6).

5.5 Qualitative Observations

Over three months of daily use (observational, not a controlled study):

1. **Session rediscovery**: Forgotten sessions containing relevant decisions were surfaced through keyword search and topic clustering.
2. **Temperature as triage**: The hot/warm/cold/frozen classification provided an intuitive prioritization mechanism for the browsing interface.
3. **Clone utility**: Clone-to-thread was most useful for multi-day tasks spanning multiple sessions. Turning-point extraction captured debugging breakthroughs that would otherwise require re-reading long transcripts.

6 Limitations

Turning-point extraction is heuristic and unevaluated for precision. The keyword-based detection produces false positives (e.g., "actually" in non-pivot contexts) and false negatives (insights without marker keywords). The ablation measures yield but not accuracy. An LLM-based extraction would likely achieve higher precision at the cost of inference dependency.

No formal user study. The qualitative observations are from a single-user deployment. A controlled study with multiple developers would be needed to validate utility more broadly.

Temperature thresholds are hand-tuned. The weights and thresholds were chosen by intuition. A calibration study would strengthen the design.

Topic clustering scales quadratically. The $O(n^2)$ Jaccard computation is mitigated by caching but could become slow for very large per-project corpora.

Single-developer corpus. Session patterns, vocabulary, and usage intensity may differ across developers and organizations.

7 Future Work

LLM-based summarization. Replace heuristic extraction with LLM-generated session summaries to improve clone quality.

Automated context injection. Use temperature to automatically select memories for injection into new sessions: hot always, warm when topic-relevant, cold/frozen never. This would extend temperature from a browsing triage mechanism to an active lifecycle management tool.

Turning-point precision study. Manually annotate turning points in a sample of sessions and measure precision/recall of the keyword-based detector against this ground truth.

Contradiction detection. Flag sessions that make conflicting decisions about the same concept using reference counting.

Multi-user evaluation. Deploy the system with multiple developers and measure impact on task resumption time and decision consistency.

8 Conclusion

We presented Claude Context Manager, a memory-management architecture for AI coding sessions designed using cybernetic reasoning about variety regulation. The three-tier hash gate—the system's primary contribution—achieves 99.96% cache hit rates on warm paths and 85% computation reduction when content is unchanged, across a corpus of 2,434 sessions (1.0 GB). Memory temperature provides a useful browsing triage mechanism, and turning-point extraction captures mid-session meaning shifts as an exploratory feature whose precision remains to be formally evaluated.

The core design pattern—using content hashes as boundary regulators to produce minimal-variety control signals at layer transitions—may generalize to adjacent systems that manage large, loosely structured, changeful corpora. Whether that generalization holds is an empirical question for future work.

References

Anysphere. Cursor: The AI-first code editor, 2024. https://cursor.sh.

W. R. Ashby. *An Introduction to Cybernetics*. Chapman & Hall, London, 1956.

Y. Bar-Yam et al. A formal definition of scale-dependent complexity and the multi-scale law of requisite variety. *Entropy*, 27(8):835, 2025.

S. Beer. *The Heart of Enterprise*. Wiley, 1979.

I. Bider and A. Jalali. Applying Ashby's law of requisite variety to software systems. In *BIR 2016 Workshops*, CEUR-WS, 2016.

M. Boisot and B. McKelvey. Complexity and organization-environment relations: Revisiting Ashby's law of requisite variety. In *The SAGE Handbook of Complexity and Management*, pages 279–298. SAGE Publications, 2011.

H. Chase. LangChain, 2022. https://github.com/langchain-ai/langchain.

M. Codoban, S. S. Jalali, C. Bird, J. Czerwonka, and P. Devanbu. Software history under the lens: A study on why and how developers examine it. In *Proceedings of ICSME 2015*, pages 1–10. IEEE, 2015.

Continue. Continue: Open-source AI code assistant, 2024. https://continue.dev.

R. Espejo and A. Reyes. *Organizational Systems: Managing Complexity with the Viable System Model*. Springer, 2011.

T. Fritz and G. C. Murphy. Using information fragments to answer the questions developers ask. In *Proceedings of ICSE 2010*, pages 175–184. ACM, 2010.

Y. Gao et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2024.

Letta. Letta: The open-source framework for building stateful AI agents, 2024. https://github.com/letta-ai/letta.

P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

J. Liu. LlamaIndex, 2022. https://github.com/run-llama/llama_index.

G. C. Murphy, M. Kersten, and L. Findlater. How are Java software developers using the Eclipse IDE? *IEEE Software*, 23(4):76–83, 2006.

OpenAI. Memory and new controls for ChatGPT, 2024. https://openai.com/blog/memory-and-new-controls-for-chatgpt.

C. Packer et al. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 1948.

L. Wang et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.

W. Zhong et al. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of AAAI 2024*, 2024.

Zep AI. Zep: Long-term memory for AI assistants, 2024. https://www.getzep.com.

A Running Example

To make the system concrete, we trace a single session through the pipeline.

Input. A JSONL session file (847 KB, 312 events) from a debugging session where the developer asked Claude Code to investigate failing tests.

Step 1: Indexing. The JSONL is streamed. Extracted metadata:

- 42 user messages, 38 assistant messages (80 total)
- Tools used: Read, Edit, Bash, Grep (4 unique)
- Duration: 47 minutes
- Classification: **standard** (80 messages, interactive)
- Importance: **62.3** (high engagement, 4 tools, file operations)

Step 2: Turning-point extraction (during clone). Two turning points detected:

- [PIVOT] User message #28: "actually, the issue might not be in the test—check the migration script instead"
- [BREAKTHROUGH] Assistant message #31: "Found it—the migration was using INTEGER instead of BIGINT, causing silent overflow on IDs above 2^{31}"

Step 3: Cache behavior. On the next API request (30 seconds later), the session's mtime has not changed → Tier 1 skip ($O(1)$). On the next day, a backup tool touches the file → mtime changes → Tier 2 hash comparison → content identical → skip re-index.

Step 4: Temperature. After being referenced in two subsequent sessions and cloned into a thread file: recency = 40 (referenced today), connectivity = 6 (2×3), importance = 18.7 (62.3×0.3) → **score 64.7** → **warm**.

B Test Coverage

The implementation includes 199 automated tests (2,111 lines of test code):

- 53 security tests (path traversal, SQL injection, input validation)
- 103 API and service tests (full CRUD on all endpoints)
- 32 variety engineering tests (hashing, temperature, caching, reference counting)
- 11 additional integration tests

All tests pass in under 2.4 seconds.